CS3210: Operating Systems

Lecture 6 — Virtual Memory: The Basics

John Kim
Georgia Institute of Technology

February 3, 2026

Contents

1 Introduction: Why Virtual Memory? 2

2 x86 Address Translation: The Big Picture 2

3 Segmentation Revisited: GDT, LDT, and Flat Segments 2
3.1 Global and Local Descriptor Tables . . . 3
3.2 Flat Segmentation in xv6 . . . 3
3.3 Segmentation vs. Paging . . . 3

4 Two-Level Paging on x86 4
4.1 The Page Directory (PD) . . . 4
4.2 The Page Table (PT) and Page Table Entries (PTEs) . . . 4
4.3 Complete Translation Walk . . . 4

5 PTE and PDE Flag Bits 5

6 Bootstrapping Paging in xv6 5
6.1 Initialization Phases: kinit1 and kinit2 . . . 6

7 The xv6 Virtual Address Space Layout 6

8 Physical Page Management in xv6 7
8.1 xv6's Page Sharing Limitation . . . 7

9 Page Table Management in xv6 8
9.1 The Kernel Memory Map: kmap . . . 8
9.2 setupkvm: Building the Kernel Page Table . . . 8
9.3 mappages: Installing PTEs . . . 8
9.4 walkpgdir: Software Page Walk . . . 9

10 Multilevel Paging: Design Tradeoffs 10
10.1 Why Not Single-Level Paging? . . . 10
10.2 Benefits of Two Levels . . . 10

10.3 Single-Level 4 MB Paging 10
10.4 Overheads of Multilevel Paging 10
10.5 Four-Level Paging in x86_64 10

11 The Memory Management Unit and TLB **11**
11.1 The MMU and Translation Lookaside Buffer 11
11.2 TLB Flush on Process Switch 11

12 Applications and Limitations of Paging **11**
12.1 What Paging Enables 11
12.2 xv6's Page Sharing Limitation 12
12.3 Research Perspective 12

13 Summary **12**

14 References **13**

1 Introduction: Why Virtual Memory?

Virtual memory is the technique by which the operating system presents each process with the illusion of its own private, contiguous address space, even though many processes share a single physical RAM. It is one of the most consequential abstractions in systems design, enabling four distinct capabilities that the hardware alone cannot provide.

1. **Isolation**: each process's virtual address space is separate from every other's. A misbehaving process cannot read or corrupt another process's memory because the mappings in its page tables simply do not include the other process's physical pages. This directly realizes the isolation guarantees introduced in Lecture 4.
2. **Resource multiplexing**: virtual memory makes one physical address space appear as many independent virtual address spaces. The hardware multiplexes physical RAM among all running processes under OS control, so each process sees a contiguous private address space starting at virtual address 0.
3. **Overcoming resource limits**: a process's virtual address space may be *larger* than the physical RAM installed in the machine. Pages that are not currently needed can be *swapped* to disk, brought back on demand. This is why a 64-bit process can allocate far more virtual memory than the machine physically has.
4. **Protection**: by controlling the permission bits stored in each page table entry, the OS can enforce fine-grained read/write/execute policies on every page. A stack page can be marked non-executable; a read-only code segment cannot be modified.

This lecture focuses on the mechanics of how virtual-to-physical address translation actually works on x86, with particular attention to xv6's implementation.

2 x86 Address Translation: The Big Picture

On x86, a CPU-generated address passes through *two* sequential translation stages before it reaches physical RAM.

1. **Segment translation**: the CPU takes a *logical address* consisting of a 16-bit **segment selector** and a 32-bit **offset**. The segment selector indexes a descriptor table (GDT or LDT) to obtain a base address, which is added to the offset to produce a **linear address**.
2. **Page translation**: the linear address is fed into the paging unit, which walks the page table hierarchy stored in memory to produce the final **physical address**.

The *virtual address* that programmers reason about is the **linear address**—the 32-bit quantity produced after segment translation and consumed by the paging unit. On modern x86 systems (including xv6) all segment bases are set to zero, so the logical offset and the linear address are numerically identical; segmentation effectively disappears. The serious isolation work is done entirely by paging.

3 Segmentation Revisited: GDT, LDT, and Flat Segments

Segmentation was the original x86 memory protection mechanism. It relies on two in-memory tables.

3.1 Global and Local Descriptor Tables

The **Global Descriptor Table (GDT)** is a system-wide array of *segment descriptors*; its physical address is loaded into the `GDTR` register. Every process on the machine shares the same GDT. The **Local Descriptor Table (LDT)** is a per-process descriptor table, but it is used very rarely in modern operating systems.

Each segment descriptor encodes three things: a **base** (the physical address where the segment starts), a **limit** (the physical address where it ends), and **flags** including the *Default Privilege Level* (DPL). Segmentation registers (`CS`, `DS`, `SS`, etc.) hold 16-bit selectors that index into the GDT or LDT as appropriate. The instruction pointer register `EIP` uses the **Code Segment (`CS`)** register; the stack pointer `ESP` uses the **Stack Segment (`SS`)** register; and general data loads/stores use the **Data Segment (`DS`)** register.

3.2 Flat Segmentation in xv6

During the boot sequence (`bootasm.S`) xv6 installs a minimal bootstrap GDT with three entries: a null descriptor, a code segment, and a data segment, both with base `0x0` and limit `0xffffffff`:

Listing 1: Bootstrap GDT in bootasm.S

```
gdt:
  SEG_NULLASM                                # null seg
  SEG_ASM(STA_X|STA_R, 0x0, 0xffffffff)    # code seg
  SEG_ASM(STA_W,        0x0, 0xffffffff)    # data seg
```

Once the full kernel is running, `seginit()` replaces this with a four-entry GDT per CPU covering kernel code (`SEG_KCODE`), kernel data (`SEG_KDATA`), user code (`SEG_UCODE`), and user data (`SEG_UDATA`):

Listing 2: seginit() in vm.c

```
void seginit(void) {
    struct cpu *c = &cpus[cpunum()];
    c->gdt[SEG_KCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, DPL_KERN);
    c->gdt[SEG_KDATA] = SEG(STA_W,       0, 0xffffffff, DPL_KERN);
    c->gdt[SEG_UCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, DPL_USER);
    c->gdt[SEG_UDATA] = SEG(STA_W,       0, 0xffffffff, DPL_USER);
    lgdt(c->gdt, sizeof(c->gdt));
}
```

All four segments share base `0x0` and limit `0xffffffff`. This is **flat segmentation**: every logical address translates to an identical linear address, so segmentation contributes nothing to isolation. The DPL fields (`DPL_KERN` = 0, `DPL_USER` = 3) are used for privilege checks, but actual memory protection is left entirely to paging.

3.3 Segmentation vs. Paging

Segmentation's one advantage is that it avoids the per-access overhead of a page table walk. Paging's advantages, however, are decisive: it avoids external fragmentation (all physical frames are equal size), it allows very large virtual address spaces, and it provides much finer-grained multiplexing of physical memory. Paging is therefore the better *isolation and protection* mechanism, and modern operating systems rely on it almost exclusively.

4 Two-Level Paging on x86

The x86 32-bit paging hardware partitions the 32-bit linear address into three fields:

$$\underbrace{b_{31} \ldots b_{22}}_{\text{10 bits: Dir}} \quad \underbrace{b_{21} \ldots b_{12}}_{\text{10 bits: Table}} \quad \underbrace{b_{11} \ldots b_{0}}_{\text{12 bits: Offset}}$$

These three fields drive a two-level lookup in physical memory, ultimately resolving the linear address to a *physical address* composed of a 20-bit **Physical Page Number (PPN)** concatenated with the 12-bit offset.

4.1 The Page Directory (PD)

The root of the page table hierarchy is the **Page Directory (PD)**, a 4 KB array of 1024 **Page Directory Entries (PDEs)**. The physical address of the PD is stored in the **CR3** control register. Critically, *CR3 holds a physical address*, not a virtual one: the paging unit has not yet been activated when CR3 is first loaded during boot, and even when paging is active, CR3 must be resolvable without translation.

Each PDE is a 32-bit quantity whose upper 20 bits are the PPN of a second-level **Page Table (PT)**, and whose lower 12 bits are flags. The 10-bit "Dir" field of the linear address selects which of the 1024 PDEs to consult ($2^{10} = 1024$ entries). The PD occupies exactly one 4 KB physical frame—a convenient size since a single frame is the unit of physical allocation.

4.2 The Page Table (PT) and Page Table Entries (PTEs)

The PDE retrieved from the PD points to a second-level Page Table, also a 4 KB array of 1024 **Page Table Entries (PTEs)**. The 10-bit "Table" field of the linear address indexes into this PT ($2^{10} = 1024$ entries). Each PTE's upper 20 bits give the PPN of the final physical memory frame, and the lower 12 bits are again permission and status flags.

4.3 Complete Translation Walk

Putting the two levels together, a full virtual-to-physical translation proceeds in five steps:

1. Load the physical address of the Page Directory from **CR3**.
2. Use the 10 most-significant bits of the linear address ("Dir") to *index into the PD* and retrieve the PDE.
3. Use the PPN in the PDE to locate the Page Table in physical memory.
4. Use the next 10 bits of the linear address ("Table") to *index into the PT* and retrieve the PTE containing the PPN of the physical frame.
5. Concatenate that PPN with the 12-bit "Offset" from the linear address to form the final physical address.

This two-level walk requires **two memory accesses** for every address translation (one PD lookup + one PT lookup), which is confirmed by the uniform class response of 100%. A k-level page table requires k memory accesses per translation in the worst case.

5 PTE and PDE Flag Bits

Both PDEs and PTEs pack permission and status information into their lower 12 bits. The most important flags are:

Bit	Name	Meaning
0	P (Present)	Page or page table is in physical memory
1	W (Writable)	Page may be written; clear = read-only
2	U (User)	Accessible from ring 3; clear = kernel only
3	WT	Write-through caching (vs. write-back when 0)
4	CD	Cache disabled for this page
5	A (Accessed)	Set by hardware on any read/write
6	D (Dirty)	Set by hardware on any write (PTE only)
7	PS (Page Size)	PDE only: 0 = 4 KB page, 1 = 4 MB page (superpage)
8	G (Global)	Page not flushed from TLB on CR3 reload
9–11	AVL	Available for OS use

The **P bit** is the most critical: if it is clear, any access to that virtual address generates a *page fault*. The OS handles page faults to implement swapping, copy-on-write, and demand paging. The **U/W bits** implement the coarse-grained isolation guarantees: kernel pages are mapped with U=0 (inaccessible to user processes) and the `read-only` code segment has W=0.

6 Bootstrapping Paging in xv6

There is a chicken-and-egg problem at boot time: to enable paging we must load a page table address into CR3, but to construct a page table we normally need to allocate physical pages—which requires *some* form of address translation to be operational first.

xv6 resolves this bootstrap problem with a **statically allocated boot page directory** called `entrypgdir`, defined in `main.c`:

Listing 3: entrypgdir in main.c — the xv6 bootstrap page directory

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
    // Map VA's [0, 4MB) to PA's [0, 4MB)
    [0] = (0) | PTE_P | PTE_W | PTE_PS,
    // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
    [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```

Because the `PTE_PS` flag is set, each of these two entries is a **4 MB superpage**—a single-level mapping that requires no second-level page table. The `Dir` field is used directly as a 10-bit index into the PD, and the remaining 22 bits of the linear address are used as a byte offset within the 4 MB region.

The two entries in `entrypgdir` create two mappings, both pointing to the same first 4 MB of physical RAM:

- **Virtual [0, 4MB) → Physical [0, 4MB)**: an identity mapping used during the first moments after paging is enabled, when the CPU's `EIP` still holds a low physical address.

- **Virtual [KERNBASE, KERNBASE+4MB) → Physical [0, 4MB)**: the high-memory mapping through which the kernel will execute once it relocates to `KERNBASE` (`0x80000000`).

The `entry.S` assembly loads this directory into CR3 and then sets the `PG` and `WP` bits in CR0 to enable paging:

Listing 4: Loading entrypgdir and enabling paging in entry.S

```
    movl    $(V2P_WO(entrypgdir)), %eax
    movl    %eax, %cr3          # set page directory
    movl    %cr0, %eax
    orl     $(CR0_PG|CR0_WP), %eax
    movl    %eax, %cr0          # turn on paging
```

Note that `V2P_WO` converts the virtual address of `entrypgdir` to a physical address (by subtracting `KERNBASE`), because CR3 must hold a physical address.

6.1 Initialization Phases: kinit1 and kinit2

Once the bootstrap page directory is active, `main()` proceeds in two phases to build the complete physical allocator and full kernel page table:

1. **kinit1()** (in `kalloc.c`): with paging still using `entrypgdir`, kinit1 places only the physical pages that *are already mapped* by `entrypgdir` onto the free list. This gives the kernel just enough memory to allocate the full kernel page table.
2. **kinit2()**: called after `kvmalloc()` has installed a full page table on all cores, kinit2 adds all remaining physical pages (up to `PHYSTOP`) to the free list.

7 The xv6 Virtual Address Space Layout

xv6 partitions the 32-bit virtual address space at **KERNBASE = 0x80000000**. Addresses below this boundary are *user space*; addresses at or above it are *kernel space*.

Listing 5: xv6 virtual address space layout

```
+----> +--------------------+ <= 0xFFFFFFFF
|      |    free memory     |
|      +--------------------+
kernel |   kernel text/data | 4MB
space  +--------------------+ <= 0x80100000
(CPL=0)|        BIOS        |
+----> +--------------------+ <= 0x80000000 (KERNBASE)
|      |        heap        |
|      +--------------------+
 user  |       stack        |
space  +--------------------+
(CPL=3)|   user text/data   |
+----> +--------------------+ <= 0x00000000
```

The kernel segment begins at `0x80000000`. The BIOS occupies the first portion of the kernel segment (`0x80000000`–`0x80100000`), followed by the kernel text and read-only data, then kernel

data and free memory growing upward. User processes occupy the lower half of the address space from `0x00000000` to `0x80000000`.

The critical implication is that the same physical kernel pages are mapped at high virtual addresses in *every* process's page table. When a user process traps into the kernel, the CPU's instruction pointer jumps to high addresses without requiring a page table switch—the kernel is always mapped. Only the user half of the page table differs between processes.

8 Physical Page Management in xv6

xv6's physical memory allocator (in `kalloc.c`) maintains a **singly-linked free list** of available 4 KB pages. Rather than maintaining a separate data structure, xv6 embeds the `struct run` (the free-list pointer) *inside* the free page itself, at the page's first bytes. This design avoids the need for any separate allocator metadata. Because a physical page is either on the free list or owned by exactly one entity (the kernel mapping or one user process), physical page tracking is simple and binary.

`kfree()` is the key function for returning a page to the free list:

Listing 6: kfree() in kalloc.c

```
void kfree(char *v) {
    struct run *r;
    if((uint)v % PGSIZE || v < end || V2P(v) >= PHYSTOP)
        panic("kfree");
    // Fill with junk to catch dangling references
    memset(v, 1, PGSIZE);
    if(kmem.use_lock) acquire(&kmem.lock);
    r = (struct run*)v;
    r->next = kmem.freelist;
    kmem.freelist = r;
    if(kmem.use_lock) release(&kmem.lock);
}
```

Several defensive measures stand out. First, `kfree` panics if the pointer is not page-aligned, if it falls below the end of the kernel's static data, or if its physical address exceeds `PHYSTOP`. Second, the `memset(v, 1, PGSIZE)` fills the just-freed page with the value `0x01`, causing any code that still holds a dangling pointer to that page to read garbage rather than old valid data—an important debugging aid.

8.1 xv6's Page Sharing Limitation

In stock xv6, each physical page may be mapped by at most two page table entries: one in the kernel mapping (which appears in every address space) and at most one in a user mapping. There is no reference counter; physical page tracking is binary (free or owned). This design simplifies the allocator but makes *copy-on-write* and *shared libraries* effectively impossible without modification—features that Lab 2 asks students to implement.

9 Page Table Management in xv6

Building and walking page tables is the job of three functions in `vm.c`: `setupkvm`, `mappages`, and `walkpgdir`.

9.1 The Kernel Memory Map: kmap

The kernel's virtual address space is defined by the static `kmap[]` table. This table describes which regions of physical memory should be mapped at which kernel virtual addresses in *every* process's page table:

Listing 7: in vm.c]Kernel memory map kmap[] in vm.c

```
static struct kmap {
    void *virt; uint phys_start; uint phys_end; int perm;
} kmap[] = {
    { (void*)KERNBASE, 0,           EXTMEM,    PTE_W},  // I/O space
    { (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},   // kern text+rodata
    { (void*)data,     V2P(data),  PHYSTOP,   PTE_W},  // kern data+memory
    { (void*)DEVSPACE, DEVSPACE,   0,         PTE_W},  // more devices
};
```

Notice that the kernel text and read-only data segment is mapped with `perm = 0` (no `PTE_W`), making it non-writable even at ring 0. This guards against accidental corruption of kernel instructions.

9.2 setupkvm: Building the Kernel Page Table

`setupkvm()` allocates a new, zeroed page directory and then calls `mappages` for each entry in `kmap[]`:

Listing 8: setupkvm() in vm.c

```
pde_t* setupkvm(void) {
    pde_t *pgdir;
    if((pgdir = (pde_t*)kalloc()) == 0) return 0;
    memset(pgdir, 0, PGSIZE);
    if(P2V(PHYSTOP) > (void*)DEVSPACE) panic("PHYSTOP too high");
    for(k = kmap; k < &kmap[NELEM(kmap)]; k++)
        if(mappages(pgdir, k->virt,
                    k->phys_end - k->phys_start,
                    (uint)k->phys_start, k->perm) < 0) {
            freevm(pgdir); return 0;
        }
    return pgdir;
}
```

9.3 mappages: Installing PTEs

`mappages(pgdir, va, size, pa, perm)` installs PTEs to map a contiguous virtual range to a contiguous physical range. It iterates page-by-page, calling `walkpgdir` to locate or allocate the PTE, then writes the physical address and flags:

Listing 9: mappages() in vm.c

```
static int mappages(pde_t *pgdir, void *va,
                    uint size, uint pa, int perm) {
    char *a, *last;
    pte_t *pte;
    a    = (char*)PGROUNDDOWN((uint)va);
    last = (char*)PGROUNDDOWN(((uint)va) + size - 1);
    for(;;) {
        if((pte = walkpgdir(pgdir, a, 1)) == 0) return -1;
        if(*pte & PTE_P) panic("remap");
        *pte = pa | perm | PTE_P;
        if(a == last) break;
        a += PGSIZE; pa += PGSIZE;
    }
    return 0;
}
```

The `panic("remap")` guard prevents accidentally overwriting an existing mapping—a sanity check that would catch double-mapping bugs immediately.

9.4 walkpgdir: Software Page Walk

`walkpgdir` is the software equivalent of what the x86 MMU does in hardware. Given a page directory, a virtual address, and an `alloc` flag, it returns a pointer to the PTE for that address (allocating a new page table frame if necessary):

Listing 10: walkpgdir() in vm.c

```
static pte_t* walkpgdir(pde_t *pgdir,
                        const void *va, int alloc) {
    pde_t *pde;
    pte_t *pgtab;
    pde = &pgdir[PDX(va)];          // index into page directory
    if(*pde & PTE_P) {
        pgtab = (pte_t*)P2V(PTE_ADDR(*pde));
    } else {
        if(!alloc || (pgtab = (pte_t*)kalloc()) == 0)
            return 0;
        memset(pgtab, 0, PGSIZE);   // zero all PTE_P bits
        *pde = V2P(pgtab) | PTE_P | PTE_W | PTE_U;
    }
    return &pgtab[PTX(va)];         // index into page table
}
```

The macro `PDX(va)` extracts the top 10 bits; `PTX(va)` extracts the next 10 bits. `PTE_ADDR(*pde)` masks off the lower 12 flag bits to get the PPN, and `P2V` converts it to a virtual address so the kernel can use it. When `alloc = 1` and the PDE is not yet present, a new page table frame is allocated with `kalloc()`, zeroed (ensuring all PTEs start with `PTE_P = 0`), and installed in the PDE with permission flags `PTE_P | PTE_W | PTE_U`—the PDE itself is given loose permissions because the individual PTEs enforce fine-grained access control.

10 Multilevel Paging: Design Tradeoffs

The two-level design is a careful balance between several competing overheads.

10.1 Why Not Single-Level Paging?

A flat, single-level page table for a 32-bit address space with 4 KB pages would need $2^{20} = 1{,}048{,}576$ entries. At 4 bytes per entry that is **4 MB of page table per process**. With dozens of processes running simultaneously, the aggregate overhead becomes prohibitive. Furthermore, most processes use only a small portion of their 4 GB virtual address space, so the vast majority of entries would be empty and wasted.

10.2 Benefits of Two Levels

The two-level hierarchy exploits sparsity. A process that uses only a few megabytes of virtual address space needs only the page directory (4 KB, always present) plus a handful of page tables (4 KB each), rather than 4 MB. Page tables for unmapped regions of the virtual address space simply do not need to exist: the corresponding PDE has its `P` bit clear, and no second-level allocation is required.

10.3 Single-Level 4 MB Paging

When the `PS` bit is set in a PDE the directory entry directly maps a **4 MB superpage**, bypassing the second level entirely. The 32-bit linear address is then split as 10 bits (directory index) + 22 bits (superpage offset). xv6 uses this exclusively during boot (`entrypgdir`) to avoid the chicken-and-egg problem of needing allocated memory to store page tables before the allocator is running.

10.4 Overheads of Multilevel Paging

Two overhead categories apply at any number of levels:

- **Memory overhead**: the aggregate physical memory consumed by all page directory and page table frames for all processes (the "sum of all PTPs").
- **Latency overhead**: every memory access by a user process requires k additional memory accesses (one per level of the page table hierarchy) to perform the translation. Without hardware assistance, this would double or quadruple every memory operation.

10.5 Four-Level Paging in x86_64

The 64-bit x86 architecture (*IA-32e* mode) extends the two-level design to **four levels** to accommodate the larger address space. A 48-bit canonical virtual address is partitioned as 9 + 9 + 9 + 9 + 12 bits, using four levels: PML4 → PDPT → PD → PT → physical frame. Each table fits in one 4 KB frame. The same idea—bit segments are indices into tables, intermediate levels point to other tables, the final level points to the physical address—applies unchanged; there are simply more levels of indirection.

11 The Memory Management Unit and TLB

Performing two (or four) memory accesses for every single load or store instruction would be catastrophically slow. The hardware mitigates this with dedicated translation caching.

11.1 The MMU and Translation Lookaside Buffer

The **Memory Management Unit (MMU)** is the hardware unit that performs the page walk automatically, completely transparent to software. When a virtual address is presented, the MMU first consults its dedicated cache—the **Translation Lookaside Buffer (TLB)**—for a cached virtual-to-physical mapping.

On a **TLB hit**, the physical address is returned immediately, with no additional memory accesses. On a **TLB miss**, the MMU performs a full page table walk in hardware (consulting CR3, then each level of the hierarchy), writes the result back into the TLB, and delivers the physical address. If the walk discovers that the page's `P` bit is clear, the MMU raises a **page fault exception**, transferring control to the OS's fault handler.

Specialized TLB variants exist for further optimization: the **iTLB** caches translations for instruction fetches only, while the **dTLB** caches translations for data accesses only. Splitting them reduces contention between the instruction-fetch pipeline and the load/store pipeline.

11.2 TLB Flush on Process Switch

Each process has a different page directory, so the virtual-to-physical mappings in the TLB are process-specific. When the OS switches processes by loading a new value into CR3, the hardware **flushes the entire TLB**—all cached translations are invalidated. The new process's early memory accesses therefore suffer TLB misses until the TLB is warm. This TLB flush is a significant part of why context switches are expensive.

12 Applications and Limitations of Paging

12.1 What Paging Enables

Paging is a powerful substrate for a variety of OS and user-level optimizations:

- **Kernel tricks (one zero-filled page)**: rather than allocating and zeroing a new physical page for every `brk()` call that extends a process's heap, the OS can map all newly allocated virtual pages to a single shared read-only zero page. On the first write, a page fault fires and the OS allocates a real page. This is *lazy zero-page allocation.*
- **Copy-on-write (CoW) fork**: when `fork()` is called, instead of copying the entire parent address space, the OS marks all pages in both parent and child as read-only in their respective page tables. Both processes share physical pages. The first write by either process triggers a page fault, at which point the OS allocates a fresh private copy of the faulted page. CoW can make `fork()` nearly instantaneous for processes that immediately call `exec()`.
- **Memory-mapped files (`mmap`)**: file contents can be mapped into the virtual address space. File I/O is then transparently performed through loads and stores rather than explicit `read`/`write` system calls.

12.2 xv6's Page Sharing Limitation

In stock xv6, at most two entities can hold a mapping to any single physical page: the kernel's global mapping and at most one user process. There is no per-page reference counter. This simplicity makes CoW and shared library support essentially impossible without modification. Lab 2 asks students to implement copy-on-write forking and lazy zero-page allocation, both of which require adding reference counting to the physical page allocator.

12.3 Research Perspective

The power of page-based virtual memory extends to distributed systems. **Snowflock** (EuroSys 2009) used on-demand paging, copy-on-write, and selective page transfer to fork a virtual machine to an entire networked cluster in under one second. **Kaleidoscope** (EuroSys 2011) applied VM state coloring for efficient stateful VM replication in cloud environments. **On-Demand-Fork** (EuroSys 2021) extended these ideas to achieve microsecond-latency fork for memory-intensive applications.

13 Summary

- **Virtual memory motivations**: isolation, resource multiplexing (1 physical → many virtual spaces), overcoming physical limits (swapping), and protection.
- **x86 address pipeline**: Logical address → Segment translation → Linear (virtual) address → Page translation → Physical address. In xv6, all segment bases are 0 (flat), so virtual = linear.
- **Flat segmentation in xv6**: `seginit()` creates four segments all with base 0 and limit `0xffffffff`; segmentation contributes no isolation.
- **Two-level paging**: a 32-bit linear address is split 10+10+12. CR3 (physical address) → Page Directory → Page Table → physical frame. Each level is a 4 KB array of 1024 32-bit entries.
- **PDE/PTE flags**: P (present), W (writable), U (user-accessible), PS (superpage), and others.
- **CR3 holds a physical address**: the paging unit must resolve CR3 without address translation.
- **entrypgdir**: xv6's bootstrap page directory maps two 4 MB superpages (VA [0,4MB) and VA [KERNBASE, KERNBASE+4MB)) both to PA [0,4MB), solving the chicken-and-egg paging bootstrap problem.
- **xv6 address space**: KERNBASE (`0x80000000`) divides user space (below) from kernel space (above). Kernel mappings appear in every process's page table.
- **TLB**: hardware cache for VA→PA translations. TLB flush on CR3 reload is a major source of context-switch overhead.
- **xv6 physical allocator**: `kalloc`/`kfree` manage a singly-linked free list embedded inside free pages; binary ownership model (no CoW support).

- **walkpgdir / mappages / setupkvm**: the three xv6 functions that implement the software page walk, PTE installation, and kernel address space construction respectively.
- **Paging applications**: lazy zero-page allocation, copy-on-write fork, memory-mapped files.

14 References

- Russ Cox, Frans Kaashoek, and Robert Morris. *xv6: A Simple, Unix-like Teaching Operating System*. MIT, 2011. `https://pdos.csail.mit.edu/6.828/2012/xv6.html`
- Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*, Volume 3A: System Programming Guide. Chapter 4 (Paging).
- Liedtke, J. "On Micro-Kernel Construction." *ACM SOSP*. 1995.
- Lagar-Cavilla, H. A., et al. "Snowflock: Rapid Virtual Machine Cloning for Cloud Computing." *EuroSys*. 2009.
- Venkataramani, G., et al. "Kaleidoscope: Cloud Micro-Elasticity via VM State Coloring." *EuroSys*. 2011.
- Ren, K., et al. "On-Demand-Fork: A Microsecond Fork for Memory-Intensive and Latency-Sensitive Applications." *EuroSys*. 2021.

Personal Notes